

Texturarea în OpenGL II. Combinarea culorilor. Transparență

1. Utilizarea mai multor obiecte de tip textură

În laboratorul precedent s-a aplicat o singură textură unui triunghi. În shaderul fragment folosit exista o variabilă uniform declarată după modelul `uniform sampler2D texture`, dar care nu avea nici o valoare atribuită din programul sursă. În general, variabilelor uniform din shader le sunt atribuite o valoare utilizând o funcție `glUniform`. Variabilele `sampler` sunt un tip de variabile GLSL folosite pentru obiecte de tip textură. Pentru o listă a texturilor ce pot fi folosite în OpenGL, accesați linkul [https://www.khronos.org/opengl/wiki/Sampler_\(GLSL\)](https://www.khronos.org/opengl/wiki/Sampler_(GLSL)).

Utilizând funcția `glUniform1i` se poate atribui o valoare unei variabile `sampler`. Pentru obiecte de textură, această valoare reprezintă un întreg fără semn ce specifică unitatea de textură (eng. *Texture unit*). Valoarea implicită pentru o textură a unității de textură este 0. Această textură este și textura activă în mod implicit. Din acest motiv, dacă se utilizează o singură textură nu este necesară trimiterea valorii unității de textură variabilei `sampler` din shaderul fragment, aceasta fiind implicit 0.

În OpenGL există un număr maxim de unități de textură ce poate fi aflat interogând valoarea constantei `GL_MAX_COMBINED_TEXTURE_IMAGE_UNITS`. În versiunile noi de OpenGL, acest număr este minim 48.

Dacă se dorește utilizarea mai multor texture, trebuie să se specifice ce textură este activă la un moment dat. Acest lucru se specifică utilizând funcția:

```
void glActiveTexture( GLenum texture );
```

Valoarea parametrului `texture` este `GL_TEXTURE0 + i`, unde *i* este un număr aflat în intervalul `[0, GL_MAX_COMBINED_TEXTURE_IMAGE_UNITS)`. Apelul acestei funcții va face ca unitatea de textură *i* să fie cea activă la un moment dat. De exemplu, dacă se activează `GL_TEXTURE1`, unitatea de textură activă va avea valoarea 1.

De obicei, apelul funcției `glActiveTexture` va fi urmat de către apelul funcției `glBindTexture` pentru a se specifica ce textură este „legată” de unitatea de textură activă și utilizată la un moment dat în program.

Pentru a putea utiliza mai multe obiecte de tip textură, în shaderul fragment, este nevoie de mai multe variabile `sampler`, câte una pentru fiecare textură ce va fi utilizată. Dacă se utilizează două texture, variabilele `sampler` folosite vor fi de forma:

```
uniform sampler2D texture1;  
uniform sampler2D texture2;
```

Pentru aceste două variabile `sampler` este necesară transmiterea din aplicația sursă a valorilor unităților de textură corespunzătoare. După transmiterea acestor valori este necesară și legarea texturilor de unitățile de textură. Toate aceste etape sunt exemplificate în codul următor:

```
GLuint texture1ID = glGetUniformLocation(shader_programme, "texture1");  
glUniform1i(texture1ID, 0);  
  
GLuint texture2ID = glGetUniformLocation(shader_programme, "texture2");  
glUniform1i(texture2ID, 1);
```

```
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, texture1);
glActiveTexture(GL_TEXTURE1);
glBindTexture(GL_TEXTURE_2D, texture2);
```

Toți parametrii texturilor `texture1` și `texture2`, precum și încărcarea valorilor texturilor în textură trebuie efectuată înaintea acestor apeluri, după modelul din laboratorul precedent.

2. Combinarea culorilor

Mai multe efecte în redarea obiectelor (transparență, ceață, anti-aliasing) se obțin prin combinarea culorilor la nivel de pixel. Pentru validarea operației de combinare a culorilor la nivel de pixel (*blending*) se apelează funcția `glEnable(GL_BLEND)`, iar pentru invalidare funcția `glDisable(GL_BLEND)`.

Pentru combinarea culorilor se specifică factorii de combinare ai sursei și ai destinației. Acești factori sunt cvadrupleți RGBA, care se multiplică cu componentele corespunzătoare R, G, B și A ale sursei, respectiv ale destinației. Pentru o descriere matematică notăm cu (s_R, s_G, s_B, s_A) factorii de combinare ai sursei, (d_R, d_G, d_B, d_A) factorii de combinare ai destinației, (R_s, G_s, B_s, A_s) componentele culorii sursei și (R_d, G_d, B_d, A_d) componentele culorii destinației. Culoarea rezultată prin combinare are componentele:

$$(R_s s_R + R_d d_R, \quad G_s s_G + G_d d_G, \quad B_s s_B + B_d d_B, \quad A_s s_A + A_d d_A)$$

Factorii de combinare se pot selecta din mai multe valori posibile prin argumentele transmise funcției:

```
void glBlendFunc(GLenum sfactor, GLenum dfactor);
```

unde parametrii `sfactor` respectiv `dfactor` reprezintă factorul de combinare al sursei respectiv al destinației. Pentru o listă a tuturor valorilor disponibile pentru acești doi parametri, accesați linkul <https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glBlendFunc.xhtml>.

În figura 1 sunt desenate două dreptunghiuri și rezultatul combinării culorii lor. Culoarea mov (cu care este desenat dreptunghiul din dreapta) are componenta alpha 1.0 iar cea verde are componenta alpha 0.4. Valoarea 1 pentru componenta alpha a unei culori specifică faptul că un obiect desenat cu această culoare este complet opac. O valoare pentru componenta alpha mai mică decât 1 specifică faptul că acea culoare are un grad de transparență.

Deci, în figura 1 se dorește desenarea dreptunghiului semitransparent verde peste dreptunghiul opac mov. Din acest motiv, dreptunghiul mov va reprezenta culoarea destinație, iar cel verde culoarea sursă.

Se pune problema acum ce factori de combinare se folosesc pentru a obține efectul dorit. Pentru ca dreptunghiul verde este semi-transparent și are culoarea cu componenta alpha 0.4, se înmulțesc toate componentele culorii sale cu această componenta alpha. Deci pentru culoarea sursă se folosește valoarea componentei alpha a acesteia. Dreptunghiul mov va avea o contribuție la culoarea finală egală cu diferența rămasă până la valoarea 1 a componentei alpha (în cazul de față 0.6). Deci factorul de combinare pentru culoarea destinație va fi 1 – componenta alpha a sursei.

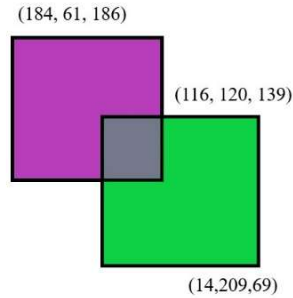


Figura 1. Combinarea culorilor pentru două dreptunghiuri cu factori de combinare 0.6 pentru dreptunghiul mov (corespunzător destinației) respectiv 0.4 pentru dreptunghiul verde (corespunzător sursei).

Pentru a obține efectul din figura 1, funcția `glBlendFunc` trebuie apelată cu parametrii `GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA`.

De notat este faptul că este importantă ordinea desenării obiectelor. Dacă s-ar desena prima dată dreptunghiul verde și după cel mov, rezultatul nu ar mai fi același, având în vedere că dreptunghiul mov este desenat complet opac. Fiecare obiect desenat corespunzător va modifica bufferul de culoare existent. În cazul celor două dreptunghiuri, se pleacă de la un buffer de culoare alb (setat prin utilizarea funcției `glClearColor`). Desenarea primului dreptunghi (cel mov) va modifica bufferul de culoare corespunzător. În momentul desenării celui de-al doilea dreptunghi (cel verde) bufferul de culoare va conține dreptunghiul mov. Deoarece acest dreptunghi este semitransparent și combinarea culorilor a fost activată, OpenGL va combina culorile acolo unde acestea se suprapun în funcție de factorii de combinare furnizați de către utilizator. Deci, dacă se dorește combinarea culorilor și se folosesc culori semitransparente, trebuie ca obiectele să fie desenate din spate în față.

3. Exerciții:

1. Completați codul din arhiva laboratorului 10 pentru a textura un triunghi utilizând două texturi, în următoarele moduri:
 - Utilizați funcția `glsl mix` pentru a combina culorile celor două texturi
 - Texturați o jumătate de triunghi cu o textură și jumătate cu cealaltă.
2. Desenați două triunghiuri colorate diferit care să se suprapună parțial (vezi arhiva din laboratorul 7 pentru desenarea unui triunghi). Triunghiul de deasupra trebuie desenat cu o culoare parțial transparentă pentru a exemplifica combinarea culorilor în OpenGL.
3. Texturați cele două triunghiuri de la exercițiul 2 cu două texturi separate. Utilizați texturi semitransparente pentru a evidenția importanța ordinii de desenare a obiectelor.